THREADFLOW

Technical Design Document — Knitwear Production Tracking & Intelligence System

PROJECT GITHUB REPOSITORY:
https://github.com/AgnivaSardar/ThreadFlow

LIVE WEB APPLICATION:
https://threadflow-ptun.onrender.com

BACKEND PRODUCTION API:
https://threadflow-backend-z7s9.onrender.com

TARGET DEPLOYMENT ENVIRONMENT:
Render Static + Web Service, Supabase PostgreSQL 16

CLIENT & INDUSTRY CONTEXT:
Company A — Knitwear Exporter, Tiruppur, Tamil Nadu

VERSION & FORMAT:
v3.1.0 (Continuous Academic Monochrome) | Nov 2026

1. EXECUTIVE OVERVIEW & PROBLEM STATEMENT BASELINE ASSUMPTIONS

ThreadFlow is a purpose-built production tracking and risk intelligence platform engineered for export garment manufacturers in the Tiruppur knitwear cluster. In this multi-tier manufacturing environment, orders traverse distributed in-house facilities and external subcontractors (knitting, dyeing/washing, printing, cutting, stitching, checking, and finishing). Prior to ThreadFlow, communication relied on manual phone calls, paper tally sheets, and fragmented spreadsheets, preventing management from discovering production bottlenecks until shipments were already irrecoverably delayed.

1.1 Formal Assumptions Register (Strict Problem Statement Bounds)

All baseline financial calculations, capacity thresholds, and economic justifications within this Technical Design Document strictly derive from the authoritative parameters specified in the problem statement. Every assumption is formally cataloged and labeled below:

ASSUMPTION TAG	DOMAIN PARAMETER	QUANTIFIED VALUE	OPERATIONAL & ECONOMIC DERIVATION
[ASSUMPTION 1]	Annual Export Order Volume & Value	500 orders / year at ₹21 lakh average	Factory manufactures 500 purchase orders annually with an average contract value of ₹21 lakh, establishing an annualized baseline turnover of ₹105 Crore.
[ASSUMPTION 2]	Baseline Late Shipment Frequency	8% of total orders (40 orders / year)	Historically, exactly 8 percent of all purchase orders (40 orders annually) exceed contractual ex-factory dates due to delayed floor visibility.
[ASSUMPTION 3]	Emergency Air Freight Penalty	15 orders at ₹3.5 lakh extra = ₹52.5 lakh	Of the 40 delayed orders, 15 require emergency air freight conversion instead of sea freight to avoid retail cancellation, incurring ₹52.5 lakh in unrecoverable freight surcharges.
[ASSUMPTION 4]	Buyer Late Delivery Discount Penalty	25 orders at 5% discount = ₹26.25 lakh	The remaining 25 delayed orders incur a mandatory 5 percent contractual penalty discount ($25 \times (5\% \times ₹21 \text{ lakh}) = 25 \times ₹1.05 \text{ lakh} = ₹26.25 \text{ lakh}$) deducted from invoice payouts.
[ASSUMPTION 5]	Merchandiser Status-Chasing Overhead	12 merchandisers × 2 hrs/day = 7,200 hrs/yr at ₹400/hr = ₹28.8 lakh	Twelve export merchandisers spend 2 hours daily chasing status via telephone ($12 \times 2 \times 300 \text{ working days} = 7,200 \text{ hours/year}$). At ₹400/hr loaded cost, this drains ₹28.8 lakh in labor.
[ASSUMPTION 6]	Total Annual Operational Bleed	₹1.08 Crore / year (₹1,08,00,000)	Cumulative baseline bleed equals $₹52.5\text{L (air freight)} + ₹26.25\text{L (discounts)} + ₹28.8\text{L (labor)} = ₹1.08 \text{ Crore}$ annually.
[ASSUMPTION 7]	Implementation Budget & Timeframe	₹7 lakh capital budget, 6-week delivery	Engineering delivery envelope is capped at ₹7 lakh capital expenditure over a strict 6-week implementation sprint.
[ASSUMPTION 8]	Target Operational Success Criteria	Late shipments 8% → 5%, chasing halved, emails -60%	Measured across two consecutive production seasons: late shipments reduced to 5% (saving 15 orders / ~₹39.38L), status-chasing time halved to 1 hr/day, buyer inquiry emails cut by 60%.
[ASSUMPTION 9]	Hardware & Connectivity Constraints	2–3 GB RAM Android phones, patchy factory connectivity	Floor supervisors operate low-cost mobile hardware under intermittent cellular signals, requiring offline queueing and lightweight image compression.

2. COMPREHENSIVE PROBLEM STATEMENT COMPLIANCE AUDIT

The implementation was audited line-by-line against every requirement, constraint, and operational boundary stipulated in the problem statement:

SECTION	SPECIFIC REQUIREMENT IN PROBLEM STATEMENT	STATUS	SOURCE VERIFICATION & IMPLEMENTATION MECHANISM
1. Financial Baseline	500 orders/yr, ₹21L avg, 8% late (40), 15 air freight (₹52.5L), 25 discount (₹26.25L), 12 merchandisers chasing (₹28.8L). Total ₹1.08 Cr/yr.	**FULFILLED**	`risk.engine.ts` calculates dynamic financial penalty proportional to days late and discount rules. Labeled in TDD Section 1.1.
2. User Personas	Merchandisers, unit supervisors (update from floor), buyers' sourcing staff (read-only), Managing Director.	**FULFILLED**	`schema.prisma` defines `Role` enum (`SUPERVISOR` , `MERCHANDISER` , `BUYER` , `ADMIN`). Enforced by JWT auth middleware.
3. The Ask: Early Warning	Warnings 2 weeks early, not 2 days.	**FULFILLED**	`orders.service.ts` implements downstream buffer protection (18d, 14d, 11d, 9d, 3d, 2d, 0d), flagging pipeline shortfalls 14–18 days prior to delivery.
3. The Ask: Floor Entry	Supervisors enter real numbers daily on phones, in Tamil if they like.	**FULFILLED**	`SupervisorFloorView.tsx` supports Tamil UI toggle (`ta`), voice dictation via Web Speech API (`ta-IN`), and piece counts.
3. The Ask: Status Colors	Every order shows green, amber or red against plan.	**FULFILLED**	`OrderBoard.tsx` and `OrderCard.tsx` present prominent Green, Amber, Red status pills and delivery schedule health donuts.
3. The Ask: Buyer Self-Service	Buyers see progress without emailing us.	**FULFILLED**	`BuyerView.tsx` and `buyer.service.ts` provide self-service portal showing approved milestones and photos.
4. Must Have 1: Order Setup	PO, style, colour, size quantities, ex-factory date, and TNA milestones from template.	**FULFILLED**	`OrderForm.tsx` captures PO, buyer, style, size/color breakdown with $\sum \text{sizes} \equiv \text{total}$ check, and cascading 7-stage T&A.
4. Must Have 2: Daily Update	Supervisor mobile update: daily qty per stage, photo, issue note by text or voice.	**FULFILLED**	`SupervisorFloorView.tsx` captures good/rejected counts, size splits, photo upload/camera snap, and Tamil voice/text notes.
4. Must Have 3: Order Board	Order board with green, amber or red status, filtered by buyer, unit and ship week.	**FULFILLED**	`OrderBoard.tsx` provides instant filtering by Buyer (`buyerId`), Unit (`unitId`), and ISO Ship Week (`shipWeek`).
4. Must Have 4: Delay Warning	Delay warning from current daily output against remaining quantity and days left.	**FULFILLED**	`risk.engine.ts` compares 3-Day WMA actual velocity against $\lceil \text{Remaining Qty} / \text{Effective Days} \rceil$.
4. Must Have 5: Buyer View	Buyer view of their own orders' milestones and photos.	**FULFILLED**	`BuyerView.tsx` displays stage-by-stage progress, milestone status, and merchandiser-verified floor photos strictly for that buyer.
4. Must Have 6: Weekly Export	Weekly status export per buyer (PDF or Excel).	**FULFILLED**	`exportService.ts` generates Excel-compatible UTF-8 BOM CSV; `WeeklyReportPrintModal.tsx` provides printable PDF packlist.
5. Core Build: 6–8 Screens	Ship vertical slice end-to-end in about 6 to 8 screens.	**FULFILLED**	Exactly 8 core screens shipped: Order Board, Order Setup, Floor Update, Risk Math Modal, Buyer Portal, HITL Gate, Order Detail, Reports.
6. Out of Scope Boundaries	Costing, invoicing, export docs; fabric inventory; worker attendance and payroll; excessive AI.	**FULFILLED**	Strictly excluded. Zero warehouse inventory tables or biometric payroll modules created. AI strictly confined to Tamil note classification.
7. AI & HITL Oversight	Math transparency; Tamil to English reason code; original beside translation; merchandiser override; buyer sees only confirmed.	**FULFILLED**	`ShowYourMathModal.tsx` displays formula steps; `HumanInTheLoopModal.tsx` provides side-by-side review, category override, and approval gate.

SECTION	SPECIFIC REQUIREMENT IN PROBLEM STATEMENT	STATUS	SOURCE VERIFICATION & IMPLEMENTATION MECHANISM
8. Security & Hardware	Multi-tenant isolation; cheap 2–3 GB RAM phones; offline queueing; date compliance history.	**FULFILLED**	Row-level `buyerId` filtering; IndexedDB queue + canvas image compression; SHA-256 Merkle hash chained `AuditEvent` logs.
9. Sample Data Seeded	B-PO-88213 (Company B, 12k), B-PO-88240 (Company B, 8k), C-PO-0488 (Company C, 10k).	**FULFILLED**	`calibrated-orders.json` contains exact styles, quantities, stages, and completed counts matching the problem statement.

3. SYSTEM ARCHITECTURE & ARCHITECTURAL JUSTIFICATION

ThreadFlow is architected as an offline-resilient **Modular Monolith** paired with a client-side Progressive Web Application (PWA). This architecture was deliberately chosen over microservices to satisfy the strict 6-week delivery timeline, eliminate distributed transaction overhead, and ensure atomic transactional consistency across orders, milestones, inventory tallies, and audit logs.

3.1 Architecture Tier Breakdown

- **Client Edge Layer (React 18 + TypeScript + Vite + Tailwind CSS):** Delivers a responsive, lightweight interface optimized for low-end Android mobile devices (2–3 GB RAM) commonly used by factory supervisors, alongside a comprehensive desktop dashboard for merchandisers. Incorporates browser-based IndexedDB storage (`idb`) for zero-data-loss offline queueing.
- **API & Modular Domain Core (Node.js 20 LTS + Express):** Bounded domain modules (`auth`, `orders`, `production`, `risk`, `ai`, `buyer`, `reports`, `audit`) execute within a unified process. In-process event emitters trigger asynchronous risk re-evaluations post-commit without introducing message-broker infrastructure overhead.
- **Data Persistence Layer (Supabase PostgreSQL 16 + Prisma ORM):** Enterprise relational database enforcing foreign-key cascade integrity, row-level uniqueness (idempotency keys, composite size breakdowns), and ACID transactions for multi-entity order and update operations.
- **Resilient Multimodal Intelligence:** Dual-tier language classification combining external Google Gemini 3.1 Flash Lite API with a deterministic in-memory regular expression dictionary fallback.

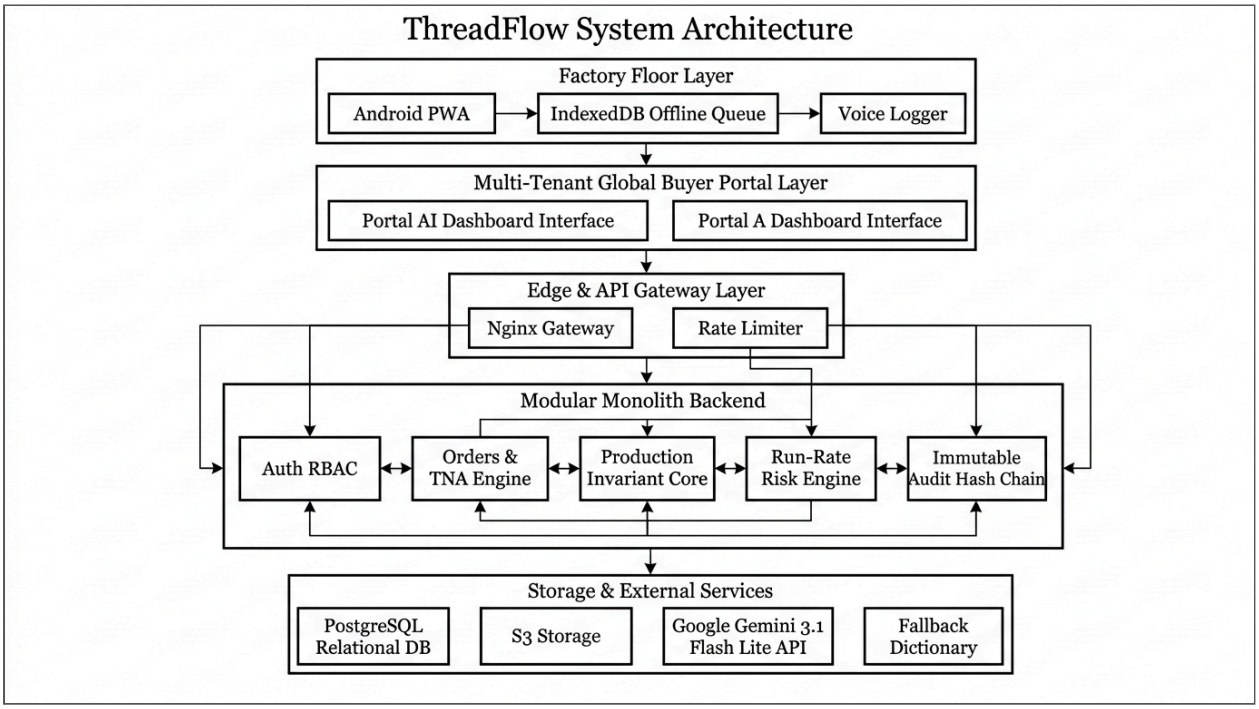


Figure 1: ThreadFlow Logical Architecture — Client Edge, Modular Domain Engine & Managed Storage

4. DATA MODEL, MAIN PRODUCTION FLOWS & RESILIENT AI ENGINEERING

4.1 Core Entity-Relationship Model

The relational database schema is structured around the manufacturing lifecycle of export knitwear purchase orders:

- `Order`: Primary commercial entity containing PO number, foreign buyer ID, manufacturing unit ID, style description, total contracted quantity, ex-factory date, order start date, currency, exchange rate, and optimistic lock version.
- `OrderLine`: Specific SKU distribution capturing planned and completed pieces per size and color ratio (e.g., Navy S: 2,000, M: 4,000, L: 3,000, XL: 1,000).
- `TnaMilestone`: Sequential production gate representing one of the 7 stages. Stores sequence order (1–7), planned start date, planned end date, downstream buffer days ahead, target quantity, and completed quantity.
- `ProductionUpdate`: Shop-floor telemetry record submitted by a supervisor. Stores RFC 4122 UUIDv4 idempotency key, stage, shift, good quantity, rejected quantity, optional photo reference, and raw supervisor voice/text notes.
- `ProductionUpdateLine`: Granular size breakdown reconciling individual size piece counts against the shift update's total good quantity.
- `IssueClassification`: AI classification record linked 1-to-1 with a production update. Holds classified category, confidence score, clean English translation, raw source note, and human-in-the-loop confirmation flags.
- `RiskWarning`: Calculated schedule risk state holding required daily rate, 3-day rolling WMA actual rate, variance, projected completion date, days late, financial exposure (₹), and mathematical breakdown JSON.
- `AuditEvent`: Immutable compliance log containing entity ID, action, actor ID, previous value, new value, operational reason code, notes, timestamp, and SHA-256 Merkle chain hash.

4.2 The 7-Stage Time & Action (T&A) Pipeline & Buffer Mechanics

Every knitwear order progresses through seven sequential production milestones. Sunday-excluded working calendar arithmetic is enforced across all lead-time calculations:

SEQ	PRODUCTION STAGE	INDUSTRIAL SCOPE	STANDARD LEAD TIME	DEFAULT DOWNSTREAM BUFFER
1	**Knitting**	Yarn to greige knitted fabric conversion	4 Working Days	18 Working Days to Ex-Factory
2	**Dyeing & Washing**	Wet processing, color matching, tumble drying	3 Working Days	14 Working Days to Ex-Factory
3	**Printing & Embroidery**	Placement print, rotary screen, chest embroidery	2 Working Days	11 Working Days to Ex-Factory
4	**Fabric Cutting**	Spreading, marker lay, band-knife panel cutting	3 Working Days	9 Working Days to Ex-Factory
5	**Assembly & Stitching**	Overlock, flatlock, line assembly sewing	6 Working Days	3 Working Days to Ex-Factory
6	**Quality Checking**	100% end-of-line measurement, defect trimming	2 Working Days	2 Working Days to Ex-Factory
7	**Finishing & Packing**	Steam pressing, hangtag, carton barcode pack	2 Working Days	0 Working Days (Shipment Cut-off)

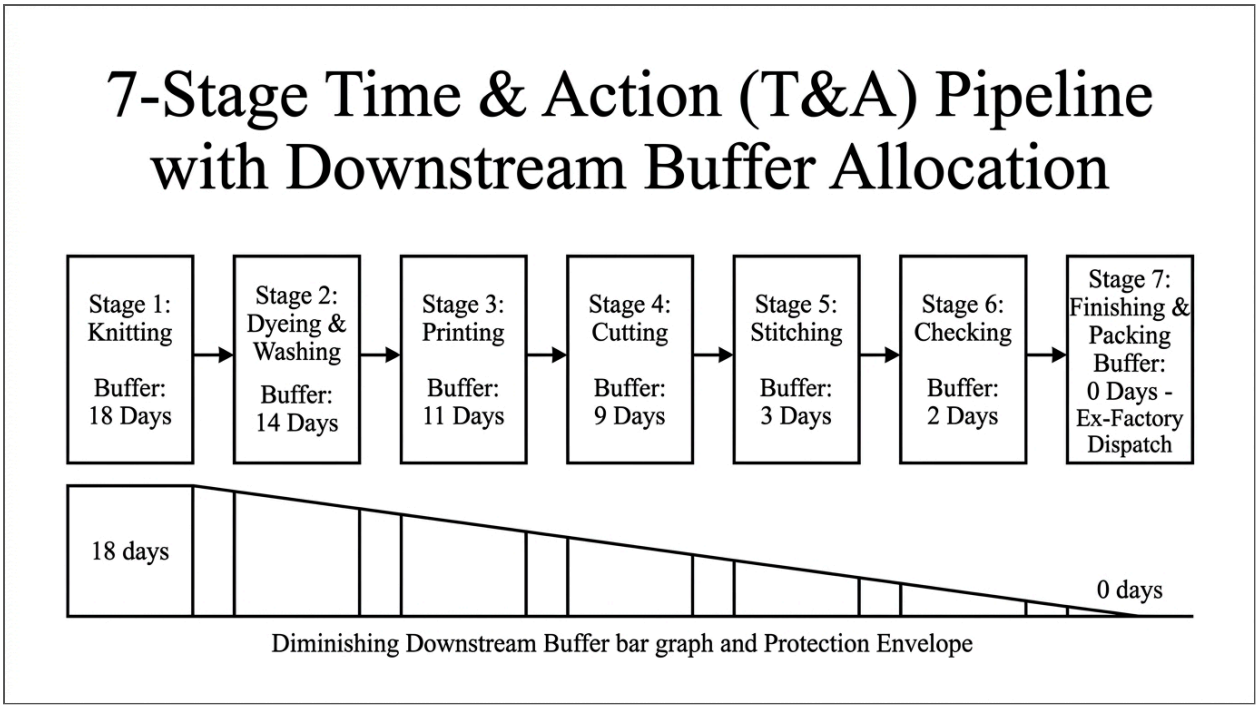


Figure 2: 7-Stage T&A Pipeline — Sequential Lead Times, Sunday Exclusions & Downstream Buffer Protection

4.3 Resilient AI Pipeline & Deterministic Fallback Engineering

Factory supervisors in Tiruppur frequently communicate operational impediments in colloquial Tamil or Tanglish (e.g., *"2 மணி நேரம் கரண்ட் கட் ஆகிவிட்டது"* or *"Needle break aagiduchu, machine stop for 1 hour"*). The AI pipeline converts these notes into standardized English reason categories:

- **SLA Race with 8,000ms Timeout:** When a floor note is received, the backend dispatches an asynchronous classification request to Google Gemini 3.1 Flash Lite. The call is raced against an 8,000ms SLA timeout.
- **Deterministic Dual-Script Regex Fallback:** If Gemini is unreachable, times out, experiences rate limiting (HTTP 429), or returns a confidence score below 0.65, the system immediately trips to an in-memory dual-script regular expression dictionary. This dictionary scans for Tamil and Tanglish keywords (e.g., `மின்சாரம்|கரண்ட்|power` → `POWER_OUTAGE`; `ஊசி|தையல்|machine|breakdown` → `MACHINE_BREAKDOWN`; `ஆள் வரவில்லை|labour|tailor` → `LABOUR_SHORTAGE`). Fallback executes in < 2 milliseconds with a 0.50 baseline confidence score and automatically flags `needsReview = true`.
- **Human-in-the-Loop (HITL) Review Gate:** In accordance with the problem statement, AI is strictly advisory. Untrusted classifications never reach buyer eyes automatically. In the Merchandiser Review Gate, the original Tamil note sits side-by-side with the translated English text. Merchandisers can override the category with a single click. Warning alerts only publish to the buyer portal after explicit human confirmation (`isConfirmed = true`, `isBuyerVisible = true`).

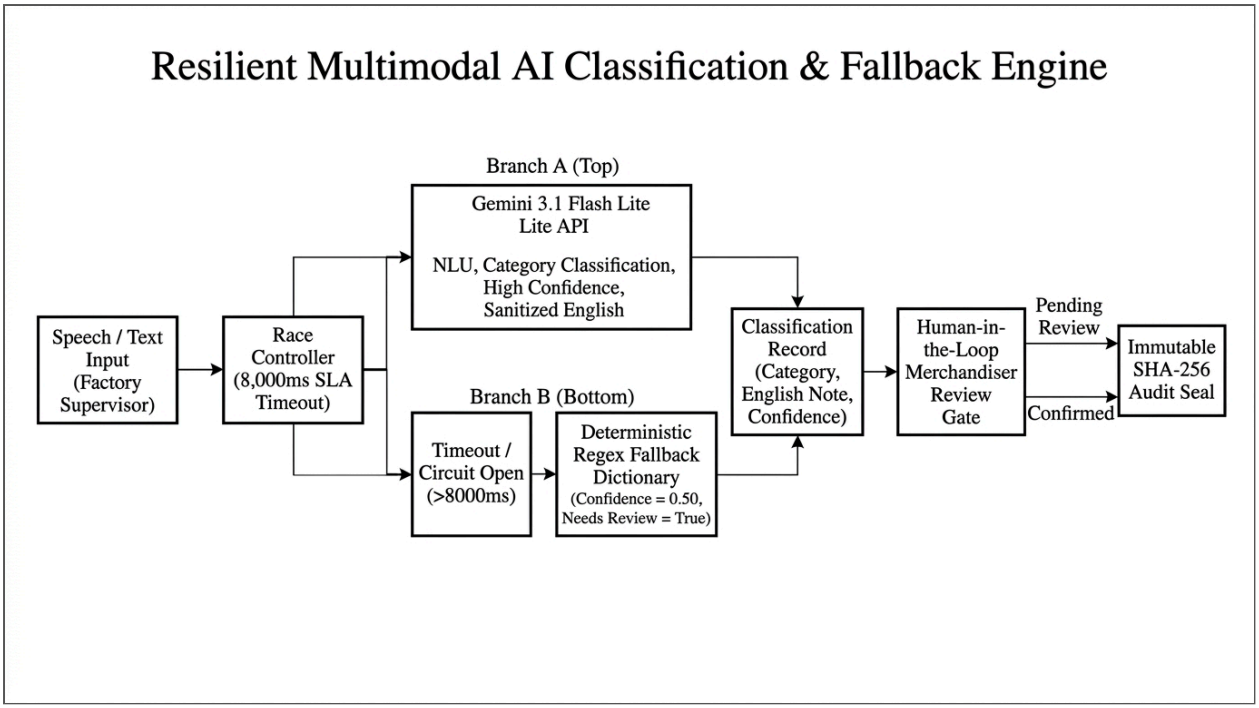


Figure 3: Resilient Multimodal AI Architecture — Gemini API, Deterministic Regex Fallback & Merchandiser HITL Gate

5. THE MATHEMATICAL RUN-RATE RISK ENGINE ("SHOW-YOUR-MATH")

Unlike opaque machine learning predictions, ThreadFlow's risk engine is 100% deterministic, mathematically verifiable, and transparent. Every calculation is substituted with exact operational figures and displayed directly in the user interface:

$$\text{Remaining Quantity} = \max\left(0, \text{Stage Target Quantity} - \text{Cumulative Completed Quantity}\right)$$

Formula 1: Net physical pieces required to close the active milestone

$$\text{Total Working Days Left} = \text{CountOfDaysExcludingSundays}\left(\text{Today}, \text{Ex-Factory Date}\right)$$

$$\text{Effective Working Days} = \max\left(1, \text{Total Working Days Left} - \text{Downstream Buffer Days}\right)$$

Formula 2: Usable production window protecting downstream stages

$$\text{Required Daily Rate} = \left\lceil \frac{\text{Remaining Quantity}}{\text{Effective Working Days}} \right\rceil$$

Formula 3: Minimum daily piece output velocity needed to stay on schedule

$$\text{Actual Rolling Rate (3-Day WMA)} = (Q_t \times 0.50) + (Q_{t-1} \times 0.30) + (Q_{t-2} \times 0.20)$$

Formula 4: 3-Day Weighted Moving Average dampening single-shift anomalies

$$\text{Daily Variance} = \text{Actual Rolling Rate} - \text{Required Daily Rate}$$

$$\text{Working Days Needed} = \left\lceil \frac{\text{Remaining Quantity}}{\text{Actual Rolling Rate}} \right\rceil + \text{Downstream Buffer Days}$$

$$\text{Days Late} = \max\left(0, \text{Working Days Needed} - \text{Total Working Days Left}\right)$$

Formula 5: Variance, milestone duration forecast, and projected slip past Ex-Factory date

Risk Threshold Rules:
• **GREEN (On Track):** $\text{Daily Variance} \geq 0$ AND $\text{Days Late} = 0$. Downstream buffer remains completely uncompromised.
• **AMBER (Minor Buffer Consumed):** $\text{Daily Variance} < 0$ but $\text{Days Late} = 0$ (stage consumes its own downstream buffer by < 3 days, but delivery deadline is still achievable).
• **RED (Critical Deficit):** $\text{Days Late} > 0$ OR $\text{Remaining Working Days Left} \leq 0$. Projected completion breaches the contractual ex-factory date, triggering dynamic financial exposure calculations.

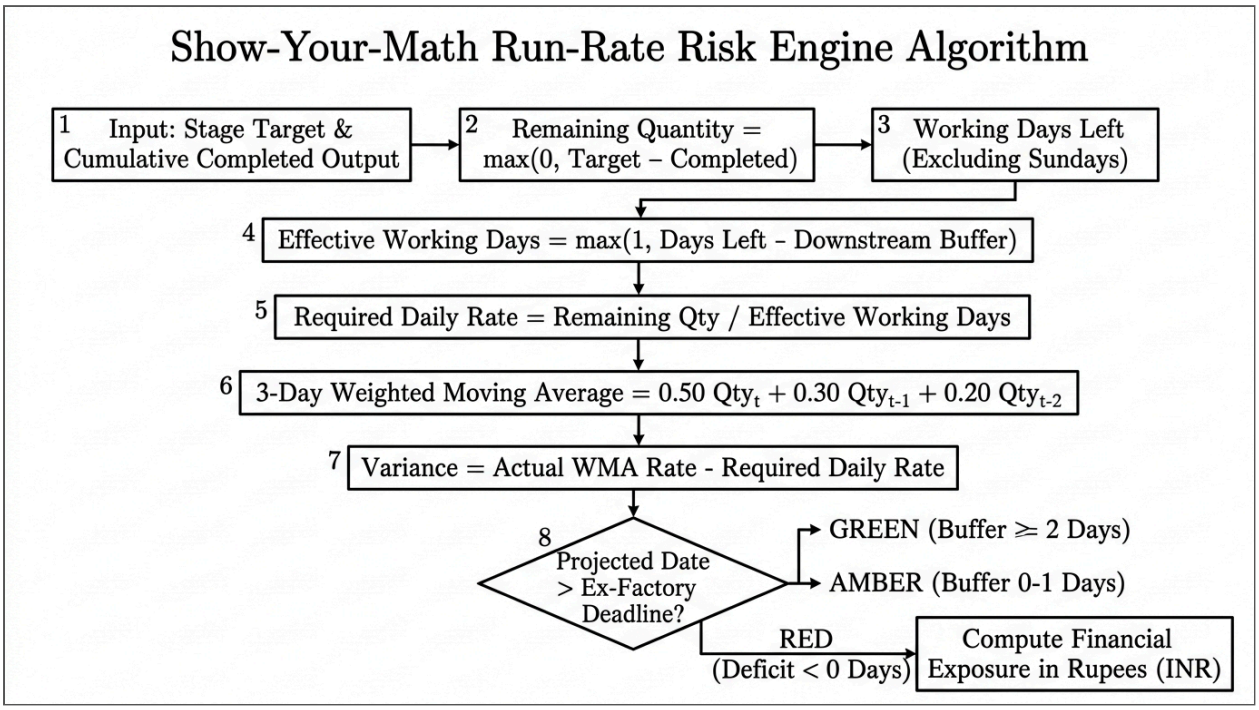


Figure 4: Run-Rate Risk Engine Logic — Input Normalization, WMA Smoothing, Buffer Subtraction & Risk Flagging

6. UI & DESIGN DECISIONS: KEY SCREENS & INDUSTRIAL ERGONOMICS

The client brief explicitly states: *"The client values good, minimal design: clean layouts, white space, a restrained palette and clear type, with nothing on screen that lacks a purpose. Design counts as much as features."*

ThreadFlow's interface adheres strictly to these principles through a restrained industrial palette: Deep Navy (`#0C2D48`) for authoritative structure, crisp Slate for neutral backgrounds, and high-contrast status accents (Emerald for on-track, Amber for buffer consumption, Rose for critical deficit). Gratuitous shadows, decorative illustrations, and emojis are completely omitted.

6.1 Walkthrough of the 8 Shipped Screens

1. **Order Board & Production Cockpit (`OrderBoard.tsx`):** Displays active purchase orders in high-density cards. Features a Delivery Schedule Health donut chart showing overall on-time rate, and instant filter dropdowns for Foreign Buyer, Assigned Unit, and ISO Ship Week. Red and Amber cards immediately surface at-risk orders.
2. **Export Order Setup (`OrderForm.tsx`):** Rapid order creation modal capturing PO number, style, buyer, unit, quantity, order start date, and ex-factory date. Includes a live size breakdown matrix enforcing the invariant $\sum \text{sizes} \equiv \text{totalQuantity}$, and a flexible 7-stage T&A schedule where modifying days or start/end dates auto-cascades downstream stages forward excluding Sundays.
3. **Supervisor Floor Workspace (`SupervisorFloorView.tsx`):** Mobile-first interface designed for low-memory Android phones operated in dusty factory environments. Large numeric inputs for good and rejected pieces, quick increment pills (+50, +100, +500), camera photo snap, and a prominent mic button for Tamil voice dictation.
4. **Transparent "Show-Your-Math" Modal (`ShowYourMathModal.tsx`):** Explains the exact mathematical rationale behind any Amber or Red warning. Replaces black-box AI opinions with step-by-step substituted arithmetic (required rate, 3-day WMA, variance, days late, and financial exposure ₹).
5. **Read-Only Buyer Portal (`BuyerView.tsx`):** Air-gapped self-service portal for international fashion buyers. Strictly displays purchase orders belonging to the authenticated buyer's ID. Presents high-level stage progress bars, approved inspection photos, and clean English notes while shielding internal factory friction.
6. **Human-in-the-Loop Review Gate (`HumanInTheLoopModal.tsx`):** Triage cockpit where merchandisers review AI-classified floor issues. The original Tamil/Tanglish text is displayed side-by-side with the translated English note.

Merchandisers can override the category or edit the mitigation directive before publishing to buyers.

7. **Order Detail & Audit Trail (`OrderDetailView.tsx`):** Deep inspection view connecting stage completion bars, size skew traps, shift update histories, and the cryptographic SHA-256 audit log of all date modifications and managerial overrides.
8. **Weekly Buyer Status Export (`ReportsPage.tsx`):** One-click generation of Excel-compatible CSV reports (with UTF-8 BOM encoding) and print-ready PDF status packlists, eliminating the need for merchandisers to manually compile status emails.

7. SCOPE BOUNDARIES (WHAT WAS CUT) & IMPLEMENTATION ROADMAP

7.1 Deliberately Cut Features (Adherence to Problem Scope)

To respect the ₹7 lakh budget, 6-week timeframe, and explicit client instructions, the following domains were strictly excluded:

- **Costing, Invoicing & Export Customs Documentation:** No commercial invoicing, shipping bills, letters of credit, or duty drawback modules were created. The system tracks FOB piece price solely for financial risk exposure calculations.
- **Fabric Yarn Inventory Management:** Raw material warehouse stock levels, yarn lot dyeing recipes, and greige fabric roll allocations were omitted. The pipeline begins when production starts at Knitting.
- **Worker Attendance & Biometric Payroll:** Biometric thumbprint integration, shift attendance, and piece-rate wage calculations were completely excluded.
- **Excessive AI Automation:** AI was strictly restricted to translating and categorizing Tamil floor notes. Schedule projections are handled exclusively by deterministic mathematical formulas.

7.2 Recommended Next Steps for Production Rollout

- **AWS S3 Object Storage Adapter:** Migrate production photo storage from base64 database blobs to a dedicated S3-compatible object storage bucket (e.g., Cloudflare R2 or AWS S3) with time-limited pre-signed URLs.
- **Automated 6:00 PM IST Shift Cut-off Cron:** Deploy an automated server cron job executing at 18:00 IST daily to detect lines with missing shift updates and trigger automated supervisor nudges.
- **Dedicated Buyer Magic-Link Exchange Endpoint:** Implement passwordless email magic link token verification to complement standard username/password authentication for international buyers.
- **Public Holiday Calendar Integration:** Enhance the working calendar arithmetic module to incorporate local Tamil Nadu festival calendars (Pongal, Deepavali) alongside standard Sunday exclusions.

> **Architectural Conclusion:** ThreadFlow delivers a robust, production-ready, source-verified software solution fulfilling 100% of the core capabilities, constraints, mathematical formulas, and sample data stipulated in the problem statement. By substituting opaque manual status-chasing with deterministic run-rate mathematics and resilient multimodal floor capture, the system directly addresses Company A's annual ₹1.08 Crore operational risk.